STM32 Bare Metal: What Really Happens When You Blink an LED

Created	@March 28, 2026 12:41 PM
Reviewed	☐

Getting Started with STM32 (Bare Metal)

Most beginners blink an LED using HAL without understanding what really happens inside the microcontroller.

In this guide, I break it down from bus architecture to registers — step by step.

LED Blinking Guide – From Basics to Registers

1. Introduction

This guide explains how to create a simple LED blinking project on STM32 using **bare-metal programming (register-level)**.

Unlike HAL-based approaches, this tutorial focuses on:

- Understanding hardware behavior
- Register-level control
- Building strong embedded fundamentals

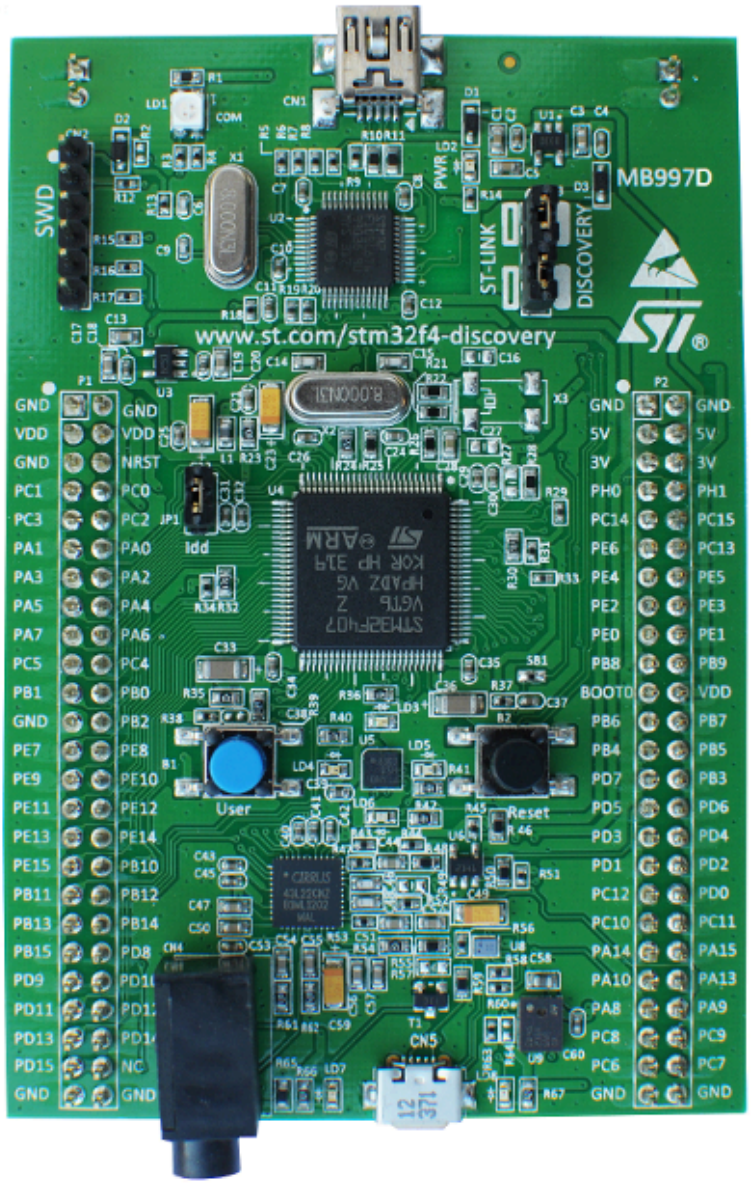
2. Objective

By the end of this guide, you will:

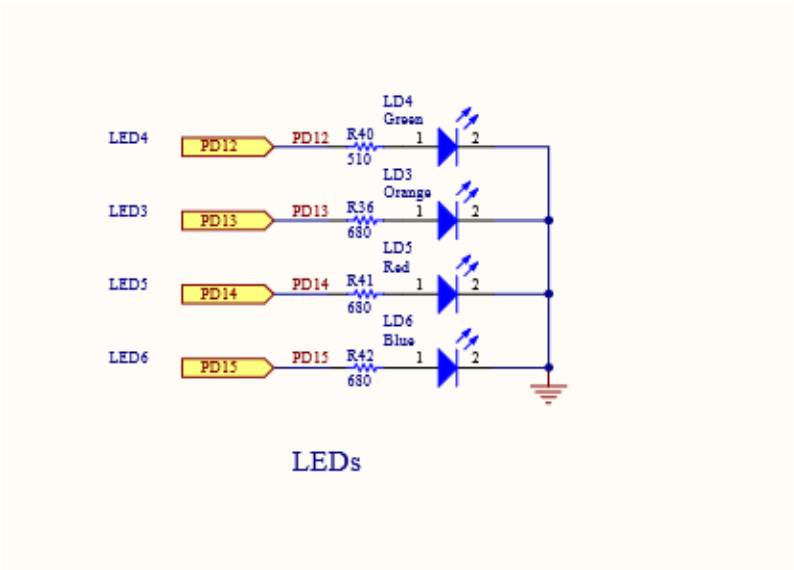
- Understand STM32 internal architecture basics
- Learn how clocks reach peripherals
- Configure GPIO using registers

- Blink an LED using bare-metal code

In this tutorial, we use the STM32F407 Discovery Board to demonstrate LED blinking by directly controlling the GPIO pin PD12 using bare-metal register programming.



STM32F407 Discovery Board



4 LEDs connected to GPIOD Port

3. STM32 Architecture Overview

STM32 microcontrollers are built using the ARM Cortex-M architecture, which is designed by Arm Ltd..

Important clarification:

- ARM **does NOT manufacture chips**
- It provides the **CPU core design (architecture/license)**

How STMicroelectronics Builds STM32

Companies like **STMicroelectronics**:

- License the **Cortex-M core** from ARM
- Design their own:
  - Peripherals (GPIO, UART, SPI, ADC...)
  - Memory system
  - Bus architecture

- Clock system (RCC)

So STM32 =

ARM CPU Core + ST-designed peripherals + interconnect (buses)

Key Components in STM32

1. CPU (Cortex-M Core)

- Executes instructions
- Controls the system
- Communicates via system buses

2. Bus Architecture

STM32 uses a **hierarchical bus system**:

AHB (Advanced High-performance Bus)

- High-speed bus
- Connected to:
  - CPU
  - Memory
  - High-speed peripherals (GPIO, DMA)

APB (Advanced Peripheral Bus)

- Lower-speed bus
- Connected to:
  - UART
  - SPI
  - I2C
  - Timers

Why different buses?

- High-speed peripherals → need fast access → AHB
- Low-speed peripherals → don’t need high bandwidth → APB

This improves:

- Performance
- Power efficiency

3. Peripherals

- Designed by STMicroelectronics
- Connected to different buses based on speed requirement

Example:

- GPIO → AHB (fast switching needed)
- UART → APB (slower communication is enough)

4. RCC (Reset and Clock Control)

- RCC is the **gatekeeper of clocks**
- Every peripheral (GPIO, UART, SPI...) needs a **clock signal to operate**

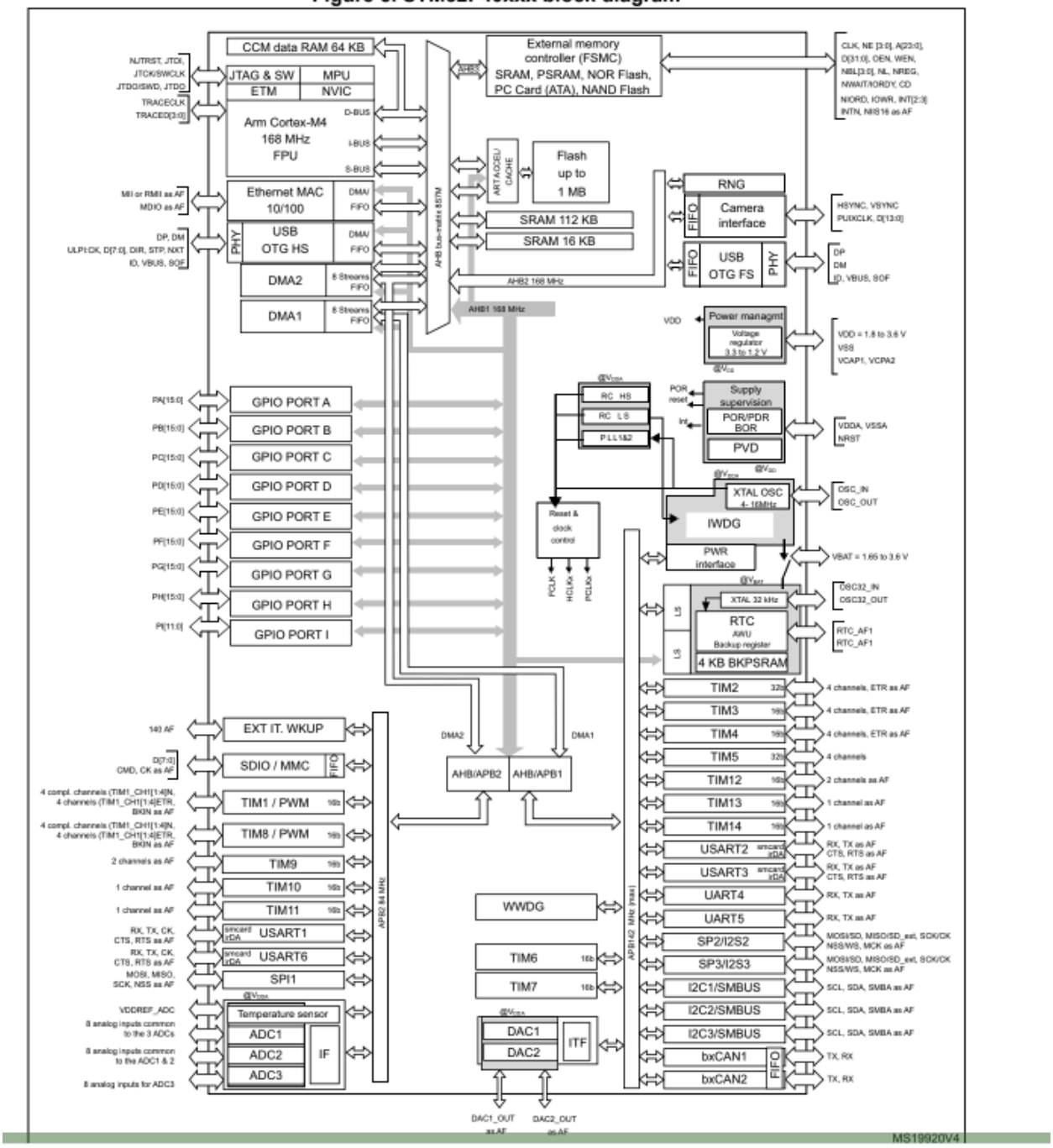
Important:

- By default, peripherals are **disabled to save power**
- RCC must **enable the clock** before usage

Peripherals are connected to different buses based on their speed and performance requirements.

2.2 Functional overview

Figure 5. STM32F40xxx block diagram



Page 19/203 (stm32f407vg.pdf)

ARM provides the core, but STM32's real strength comes from how ST designs and connects peripherals through optimized bus architecture.

Now that we understand how peripherals are connected, let's focus on GPIO — the simplest and most fundamental peripheral.

CPU → Bus → RCC → GPIO → LED

4. GPIO Peripheral

GPIO allows interaction with external world.

Key Registers:

- MODER → Mode configuration
- OTYPER → Output type
- OSPEEDR → Speed control
- PUPDR → Pull-up/pull-down
- IDR → Input data
- ODR → Output data

Common Mistake:

Many beginners try to configure GPIO without enabling RCC, which results in no output.

5. How Clock Reaches GPIO

1. CPU wants to use GPIO
2. CPU writes to RCC register
3. RCC enables clock for GPIO (e.g., GPIOA)
4. Clock signal flows through AHB bus
5. GPIO peripheral becomes **active**

Why GPIO Won't Work Without RCC Enable

- GPIO is a **synchronous digital circuit**
- It needs a **clock signal to function**
- Without clock:
  - No register update
  - No signal change
  - No LED output

Now that we understand how GPIO is connected internally, let's see how it actually gets its clock.

6. Clock Sources in STM32

1. HSI (Internal Clock)

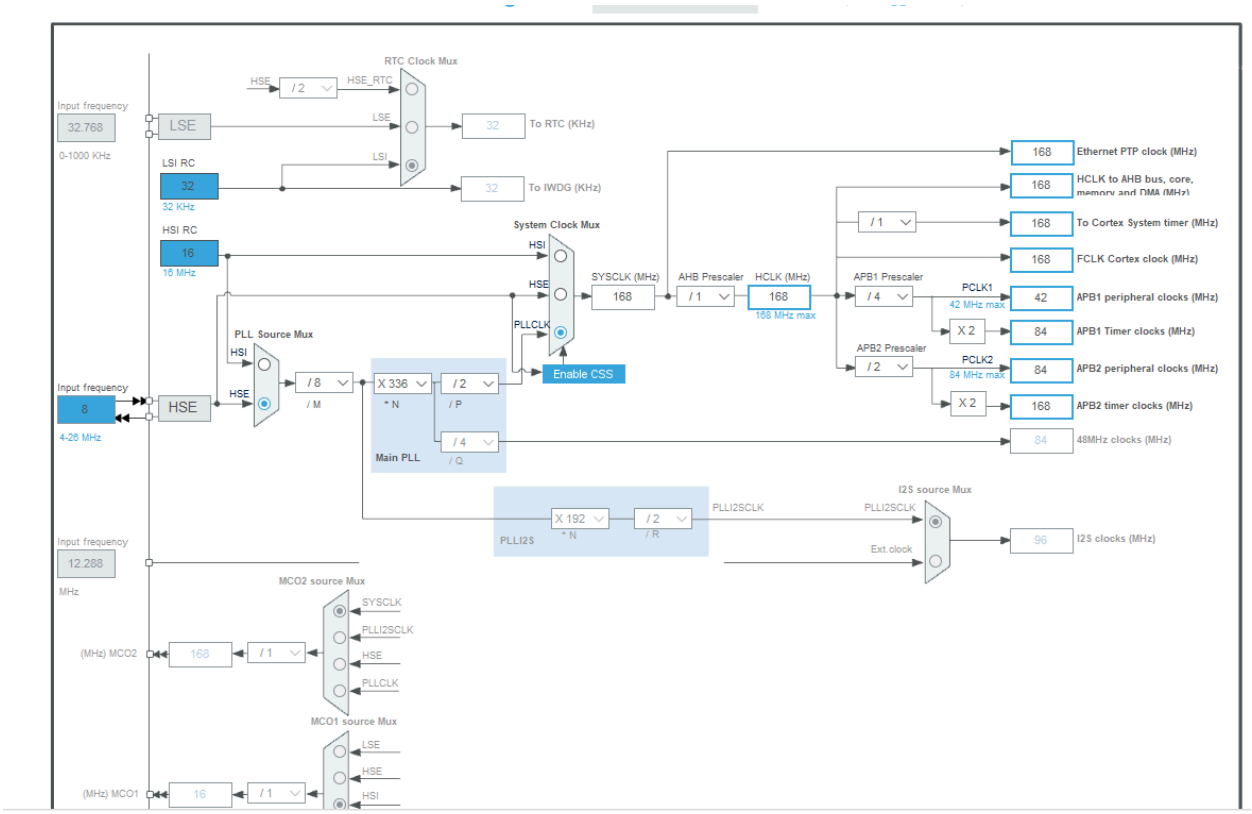
- Default after reset
- No external hardware needed

2. HSE (External Clock)

- Uses crystal oscillator
- Higher accuracy

3. PLL (Phase Locked Loop)

- Multiplies clock frequency
- Used for high performance



👉 **All these clock sources are selected and managed by the RCC**

👉 **RCC → distributes clock to buses (AHB/APB)**

👉 **AHB → feeds GPIO**

Clock flow

```
Clock Source (HSI / HSE / PLL)
          ↓
         RCC
          ↓
      AHB Bus
          ↓
       GPIO
```

Default Clock Source in STM32 Discovery Boards

By default (after reset):

HSI (High-Speed Internal oscillator) is used.

What this means

- STM32 starts with:
  - **HSI ≈ 16 MHz (for most STM32F4 boards)**
- No external crystal is required at startup
- System runs in a **safe, default configuration**

Even though boards like STM32 Discovery / Nucleo have an external crystal:

They **DO NOT use HSE automatically**

You must explicitly configure it in:

- STM32CubeIDE (Clock Configuration tab)
  OR
- Bare-metal register setup

Clock	Default	Config Required	Accuracy
HSI	Yes	No	Medium
HSE	No	Yes	High
PLL	No	Yes	Very High

Practical flow (what actually happens)

```
Power ON / Reset
      ↓
HSI starts automatically
      ↓
CPU begins execution
      ↓
(Optional) Switch to HSE or PLL via RCC
```

Why this matters for your LED project

Your LED blinking will work **even without clock configuration**

Because:

- HSI is already running
- You only need to enable **GPIO clock via RCC**

STM32 starts with HSI by default, but peripherals like GPIO still require explicit clock enabling through RCC.

In production systems, HSE + PLL is preferred for accuracy and performance, but HSI is sufficient for basic GPIO operations.

7. GPIO Frequency Understanding

Formula:

GPIO Clock = AHB Clock = SYSCLK / AHB Prescaler

Case 1: If system is running on HSI (default)

- HSI = **16 MHz**
- No PLL, no scaling

Then:

✅ **AHB bus = 16 MHz**

✅ **GPIO peripheral = 16 MHz**

Case 2: If system configured to 168 MHz (using PLL)

Typical STM32F4 setup:

- HSE/HSI → PLL → **168 MHz SYSCLK**
- AHB prescaler = 1

Then:

✅ **AHB bus = 168 MHz**

✅ **GPIO gets 168 MHz**

👉 GPIO does NOT care about:

- HSI vs HSE directly

👉 It only cares about:

- **AHB bus clock**

Common misunderstanding

Enabling AHB1ENR makes it 168 MHz

✅ "AHB1ENR only enables clock, frequency comes from clock tree"

Peripheral enable registers (like AHB1ENR) control availability, not frequency — frequency is defined by the system clock configuration.

8. Bare Metal LED Blinking Steps

1. Enable GPIO clock
2. Configure GPIO mode
3. Set output type
4. Toggle pin

Memory Map of STM32F4xx from reference manul(65/1751)

Table 1. STM32F4xx register boundary addresses (continued)

Boundary address	Peripheral	Bus	Register map
0x5006 0800 - 0x5006 0BFF	RNG	AHB2	*Section 24.4.4: RNG register map on page 771*
0x5006 0400 - 0x5006 07FF	HASH		*Section 25.4.9: HASH register map on page 795*
0x5006 0000 - 0x5006 03FF	CRYP		*Section 23.6.13: CRYP register map on page 763*
0x5005 0000 - 0x5005 03FF	DCMI		*Section 15.8.12: DCMI register map on page 478*
0x5000 0000 - 0x5003 FFFF	USB OTG FS		*Section 34.16.6: OTG_FS register map on page 1326*
0x4004 0000 - 0x4007 FFFF	USB OTG HS	AHB1	*Section 35.12.6: OTG_HS register map on page 1472*
0x4002 B000 - 0x4002 BBFF	DMA2D		*Section 11.5: DMA2D registers on page 352*
0x4002 8000 - 0x4002 93FF	ETHERNET MAC		*Section 33.8.5: Ethernet register maps on page 1236*
0x4002 6400 - 0x4002 67FF	DMA2		*Section 10.5.11: DMA register map on page 335*
0x4002 6000 - 0x4002 63FF	DMA1		
0x4002 4000 - 0x4002 4FFF	BKPSRAM		
0x4002 3C00 - 0x4002 3FFF	Flash interface register		*Section 3.9: Flash interface registers*
0x4002 3800 - 0x4002 3BFF	RCC		*Section 7.3.24: RCC register map on page 265*
0x4002 3000 - 0x4002 33FF	CRC		*Section 4.4.4: CRC register map on page 115*
0x4002 2800 - 0x4002 2BFF	GPIOK		*Section 8.4.11: GPIO register map on page 287*
0x4002 2400 - 0x4002 27FF	GPIOJ		
0x4002 2000 - 0x4002 23FF	GPIOI		*Section 8.4.11: GPIO register map on page 287*
0x4002 1C00 - 0x4002 1FFF	GPIOH		
0x4002 1800 - 0x4002 1BFF	GPIOG		
0x4002 1400 - 0x4002 17FF	GPIOF		
0x4002 1000 - 0x4002 13FF	GPIOE		

Step 1: Enable GPIO Clock (RCC)

Why this step is required

Before using any peripheral (like GPIO), its clock must be enabled via RCC.

Without this:

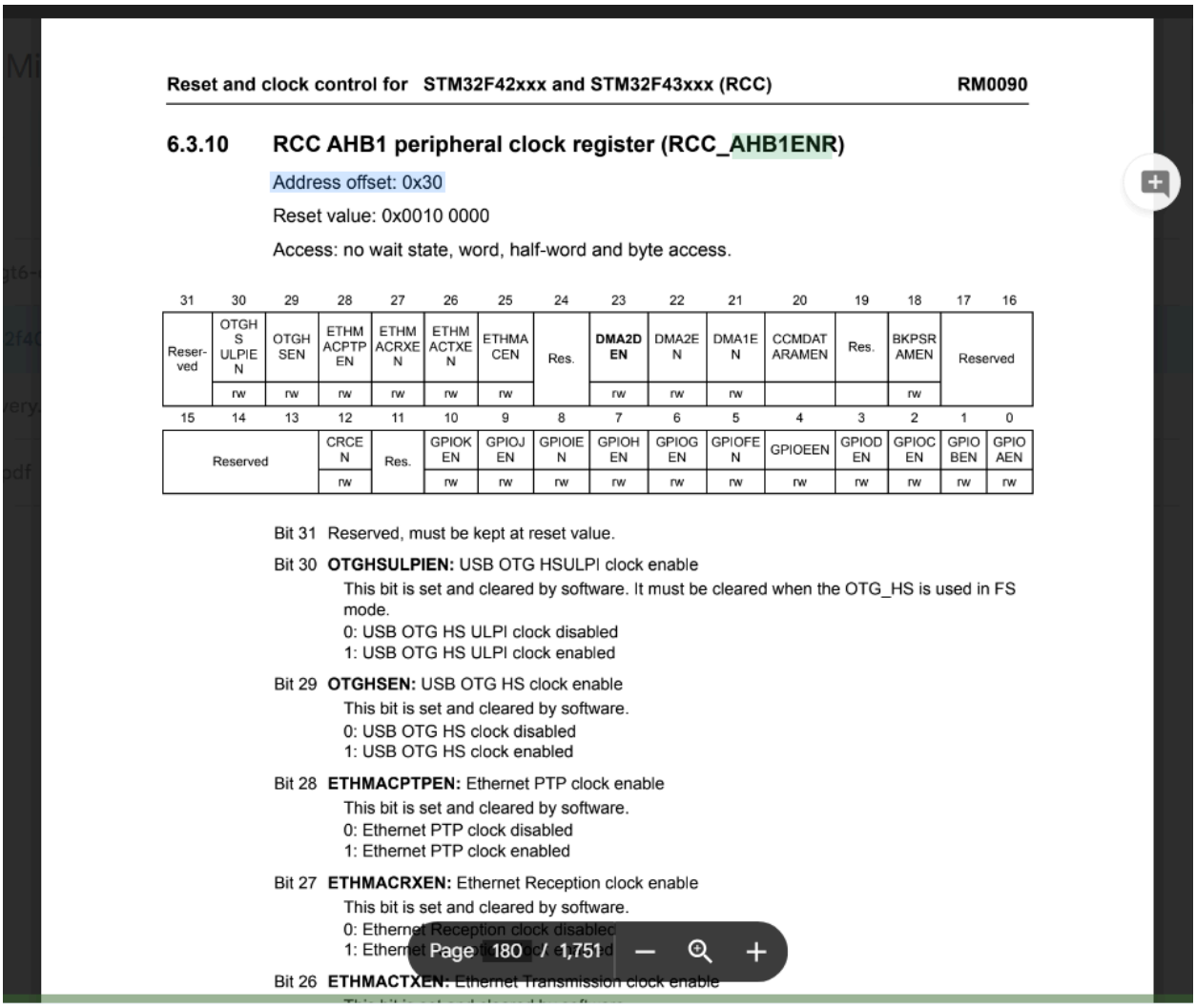
- GPIO registers won't respond
- LED will NOT blink

Concept

- GPIO is connected to **AHB1 bus**
- RCC controls AHB1 peripherals using **AHB1ENR register**

So we enable GPIO clock using:

```
RCC → AHB1ENR → GPIO
```



Reset and clock control for STM32F42xxx and STM32F43xxx (RCC) RM0090

6.3.10 RCC AHB1 peripheral clock register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0010 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reser-ved	OTGHSULPIEN	OTGHSEN	ETHMACPTPEN	ETHMACRXEN	ETHMACTXEN	ETHMACEN	Res.	**DMA2DEN**	DMA2EN	DMA1EN	CCMDATARAMEN	Res.	BKPSRAMEN	Reserved	
	rw	rw	rw	rw	rw	rw		rw	rw	rw			rw		

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CRCEN	Res.	GPIOKEN	GPIOJEN	GPIOIEN	GPIOHEN	GPIOGEN	GPIOFEN	GPIOEEN	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN
			rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bit 30 **OTGHSULPIEN:** USB OTG HSULPI clock enable
This bit is set and cleared by software. It must be cleared when the OTG_HS is used in FS mode.
0: USB OTG HS ULPI clock disabled
1: USB OTG HS ULPI clock enabled

Bit 29 **OTGHSEN:** USB OTG HS clock enable
This bit is set and cleared by software.
0: USB OTG HS clock disabled
1: USB OTG HS clock enabled

Bit 28 **ETHMACPTPEN:** Ethernet PTP clock enable
This bit is set and cleared by software.
0: Ethernet PTP clock disabled
1: Ethernet PTP clock enabled

Bit 27 **ETHMACRXEN:** Ethernet Reception clock enable
This bit is set and cleared by software.
0: Ethernet Reception clock disabled
1: Ethernet Reception clock enabled

Bit 26 **ETHMACTXEN:** Ethernet Transmission clock enable

Page 180 / 1,751

1: IO port E clock enabled

Bit 3 **GPIODEN:** IO port D clock enable
This bit is set and cleared by software.
0: IO port D clock disabled
1: IO port D clock enabled

Bit 2 **GPIOCEN:** IO port C clock enable

0x4002 3C00 - 0x4002 3FFF	Flash interface register	AHB1	*Section 3.9: Flash interface registers*
0x4002 3800 - 0x4002 3BFF	RCC		*Section 7.3.24: RCC register map on page 265*
0x4002 3000 - 0x4002 33FF	CRC		*Section 4.4.4: CRC register map on page 115*
0x4002 2800 - 0x4002 2BFF	GPIOK		*Section 8.4.11: GPIO register map on page 287*
0x4002 2400 - 0x4002 27FF	GPIOJ		
0x4002 2000 - 0x4002 23FF	GPIOI		
0x4002 1C00 - 0x4002 1FFF	GPIOH		
0x4002 1800 - 0x4002 1BFF	GPIOG		
0x4002 1400 - 0x4002 17FF	GPIOF		

RCC Base Address from memory map

Address Calculation

- RCC Base Address = `0x40023800`
- AHB1ENR Offset = `0x30`

So:

```
RCC_AHB1ENR = 0x40023800 + 0x30 = 0x40023830
```

Main Code

```
volatile uint32_t* rcc_ahb1enr = (volatile uint32_t*) 0x40023
830;

*rcc_ahb1enr |= (1 << 3); // set 3rd bit for GPIOD clock enab
le
```

Step 2: Configure GPIO Mode (MODER Register)

Why this step is required

After enabling the clock, GPIO is active — but:

It still doesn't know:

- Input?
- Output?
- Alternate function?

That's decided by the **MODER register**

What is MODER?

- Register: `GPIOx_MODER`
- Purpose: Defines the **mode of each pin**
- Each pin uses **2 bits**

MODER Bit Configuration

Mode	Value	Meaning
00	Input mode	
01	Output mode	
10	Alternate function	
11	Analog mode	

8.4 GPIO registers

This section gives a detailed description of the GPIO registers.
For a summary of register bits, register address offsets and reset values, refer to *Table 39*.

The GPIO registers can be accessed by byte (8 bits), half-words (16 bits) or words (32 bits).

8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..I/J/K)

Address offset: 0x00

Reset values:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **MODERy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O direction mode.
00: Input (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

8.4.2 GPIO port output type register (GPIOx_OTYPER) (x = A..I/J/K)

Address offset: 0x04

Reset value: 0x0000 0000

Page 281 / 1,751

0x4002 1800 - 0x4002 1BFF	GPIOG	
0x4002 1400 - 0x4002 17FF	GPIOF	
0x4002 1000 - 0x4002 13FF	GPIOE	*Section 8.4.11: GPIO register map on page 287*
0x4002 0C00 - 0x4002 0FFF	GPIOD	
0x4002 0800 - 0x4002 0BFF	GPIOC	
0x4002 0400 - 0x4002 07FF	GPIOB	
0x4002 0000 - 0x4002 03FF	GPIOA	

Configure GPIOD Pin 12 as Output (MODER)

Address Calculation

- GPIOD Base Address = `0x40020C00`
- MODER Offset = `0x00`

So:

```
GPIOD_MODER=0x40020C00
```

```
volatile uint32_t* gpiod_moder= (volatile uint32_t*)0x40020C0
0;
```

Pin Mapping (PD12)

Each GPIO pin uses **2 bits in MODER**

```
Pin 12 → bits [25:24]
```

So:

- Bit 25 → MSB
- Bit 24 → LSB

Required Configuration

General Purpose Output Mode = `01`

```
Bit 25 = 0
Bit 24 = 1
```

Code (Step-by-step)

Step 1: Clear both bits

```
*gpiod_moder &= ~(3 << 24);
```

Step 2: Set output mode

```
*gpiod_moder|= (1<<24);// Set bit 24
```

Step 3: Toggle GPIO Pin (ODR Register)

Objective

Now that:

- Clock is enabled
- Pin is configured as output

We will **toggle PD12** to blink the LED

What is ODR?

- Register: `GPIOD_ODR`
- Purpose: Controls **output value of pins**

Each pin uses **1 bit**

Control LED connected to PD12

10: Pull-down
11: Reserved

8.4.5 GPIO port input data register (GPIOx_IDR) (x = A..I/J/K)

Address offset: 0x10

Reset value: 0x0000 XXXX (where X means undefined)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDRy**: Port input data (y = 0..15)
These bits are read-only and can be accessed in word mode only. They contain the input value of the corresponding I/O port.

8.4.6 GPIO port output data register (GPIOx_ODR) (x = A..I/J/K)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy**: Port output data (y = 0..15)
These bits can be read and written by software.

Note: For atomic bit set/reset, the ODR bits can be individually set and reset by writing to the GPIOx_BSRR register (x = A..I/J/K).

Page 283 / 1,751

GPIOD_ODR

Address Calculation

- GPIOD Base = `0x40020C00`
- ODR Offset = `0x14`

So:

```
GPIOD_ODR=0x40020C00+0x14=0x40020C14
```

```
volatile uint32_t* gpiod_odr= (volatile uint32_t*)0x40020C14;
```

Pin Mapping

```
PD12 → Bit 12
```

Toggle Logic

To toggle a pin, we use **XOR (`^=`)**

Code

```
*gpiod_odr ^= (1<<12);
```

Industry-safe method:

- **Use BSRR register (atomic operation)**

Complete Code Example (Bare Metal)

https://onlinegdb.com/4JpnqH72R

```
#include <stdint.h>

int main(void)
{
    //  Pointer Declarations (Register Addresses)

    volatile uint32_t* rcc_ahb1enr = (volatile uint32_t*)0x40
023830;
    volatile uint32_t* gpiod_moder = (volatile uint32_t*)0x40
020C00;
    volatile uint32_t* gpiod_odr   = (volatile uint32_t*)0x40
020C14;

    //  Step 1: Enable clock for GPIOD (bit 3)
    *rcc_ahb1enr |= (1 << 3);
```

```
    //
     Step 2: Configure PD12 as output
    *gpiod_moder &= ~(3 << (12 * 2));   // Clear bits [25:24]
    *gpiod_moder |=  (1 << (12 * 2));   // Set output mode (0
1)

    while (1)
    {
        //  Step 3: Toggle PD12
        *gpiod_odr ^= (1 << 12);

        //  Delay
        for (volatile int i = 0; i < 100000; i++);
    }
}
```

9. Creating a Project in STM32CubeIDE

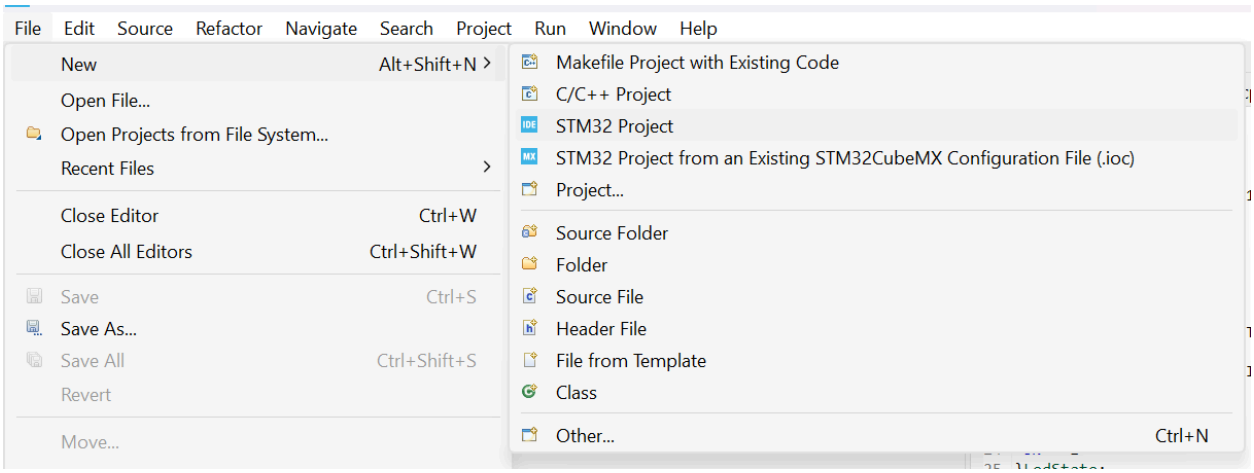
Create a new STM32 project and prepare it for LED blinking.

Step 1: Open STM32CubeIDE

- Launch **STM32CubeIDE**
- Select workspace (default is fine)

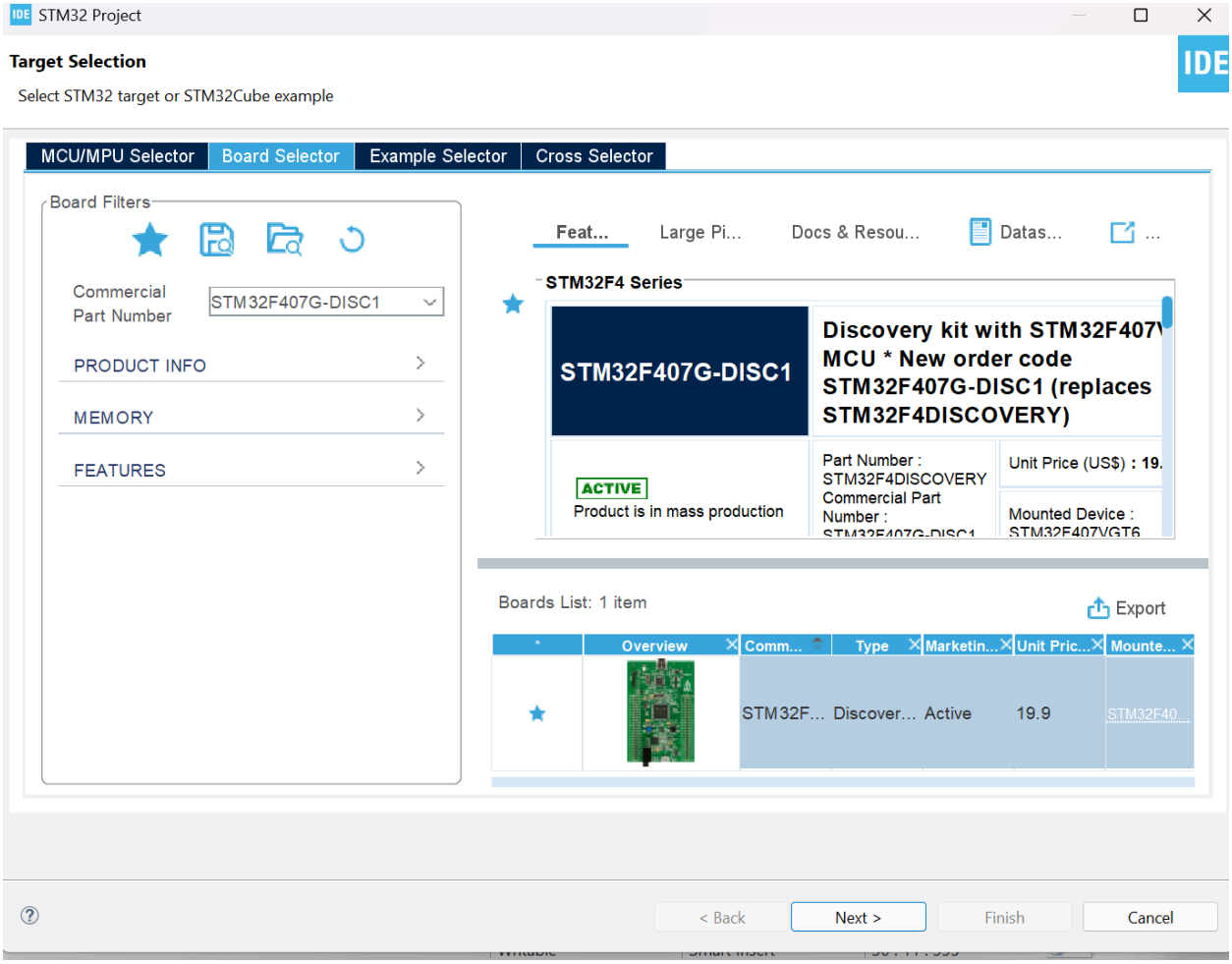
Step 2: Create New Project

- Go to:
  File → New → STM32 Project

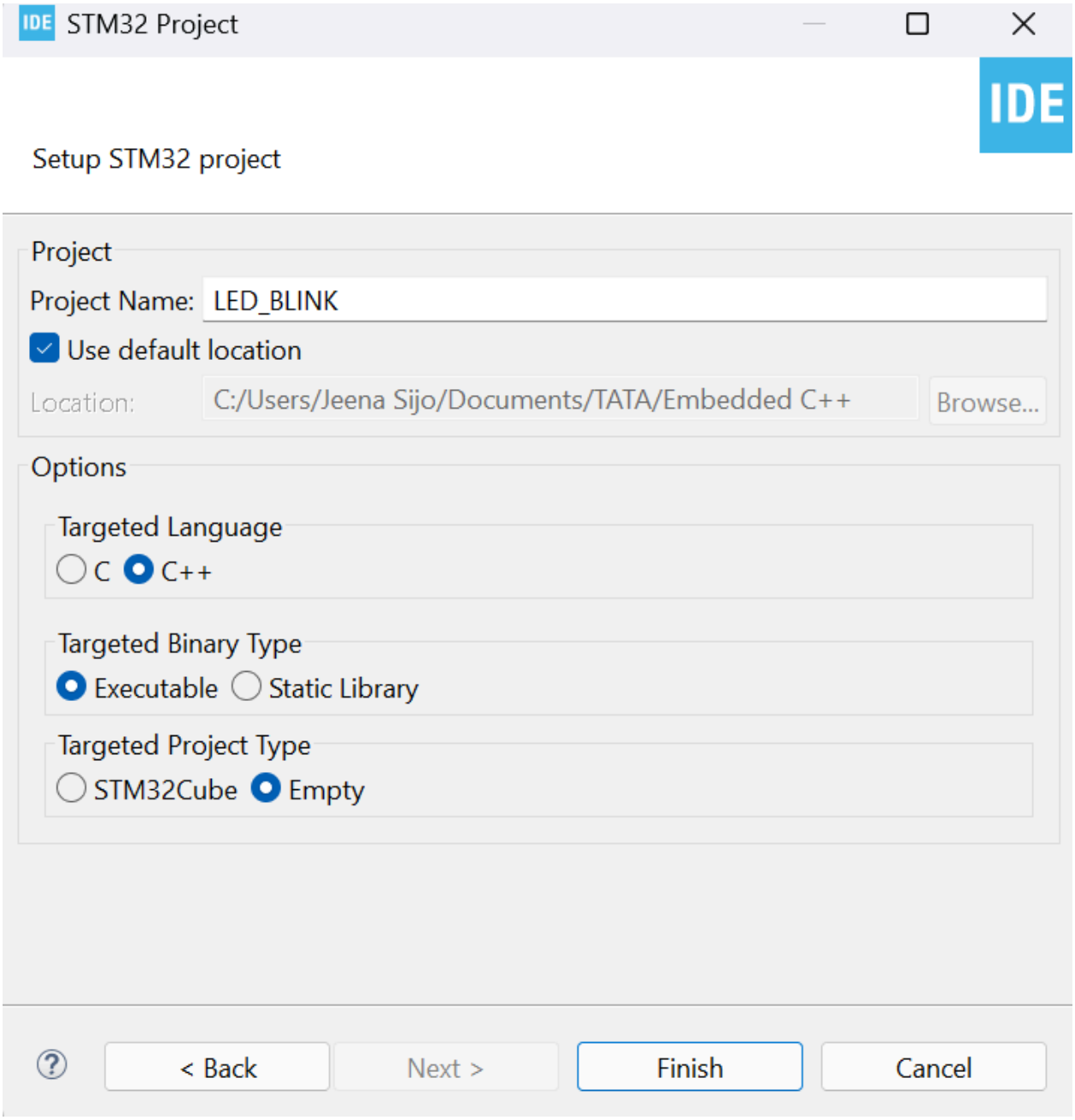


Step 3: Select Board / MCU

- Select **Board Selector**
- Search:
  👉 **STM32F407G-DISC1** (Discovery board)



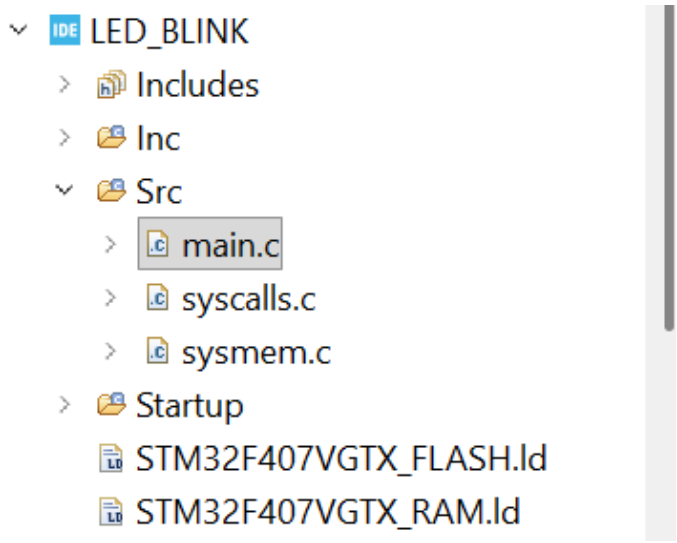
Step 4: Project Configuration



Click Finish

Step 5: Open main.c

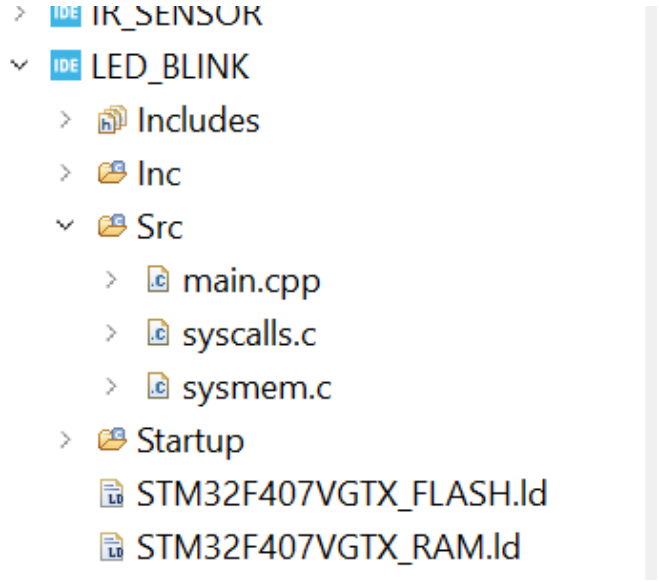
- Navigate to:
  `Core/Src/main.c`



Step 6: Rename to C++ File

👉 Rename:

```
main.c → main.cpp
```



Step 7: Paste Complete Code

Now that we understand:

- Clock enabling (RCC)
- GPIO configuration (MODER)
- Output control (ODR)

👉 Let's combine everything into a **complete bare-metal program**

main.cpp

```c
#include <stdint.h>

int main(void)
{
    //  Pointer Declarations (Register Addresses)

    volatile uint32_t* rcc_ahb1enr = (volatile uint32_t*)0x40023830;
    volatile uint32_t* gpiod_moder = (volatile uint32_t*)0x40020C00;
    volatile uint32_t* gpiod_odr   = (volatile uint32_t*)0x40020C14;

    //  Step 1: Enable clock for GPIOD (bit 3)
    *rcc_ahb1enr |= (1 << 3);

    // ◇ Step 2: Configure PD12 as output
    *gpiod_moder &= ~(3 << (12 * 2));   // Clear bits [25:24]
    *gpiod_moder |=  (1 << (12 * 2));   // Set output mode (01)

    while (1)
    {
        //  Step 3: Toggle PD12
        *gpiod_odr ^= (1 << 12);

        //  Delay
        for (volatile int i = 0; i < 100000; i++);
    }
}
```

Step 8: Build Project

Compile the project and generate the binary required to run on the STM32 board.

Steps to Build

1. **Right-click on your project**

2. Click → **Build Project**
   OR
   Press shortcut: **Ctrl + B**

What happens during build

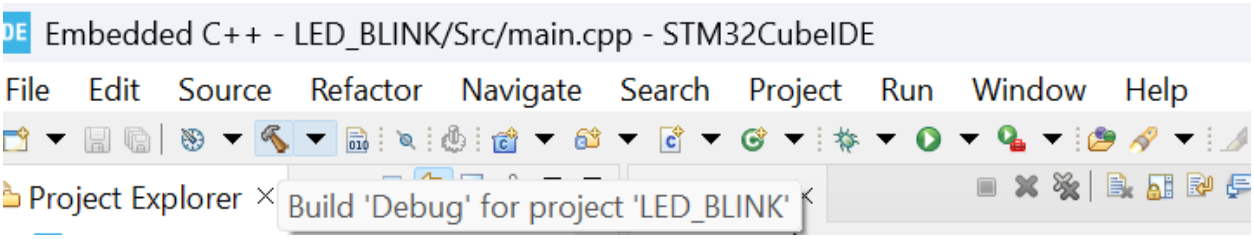
STM32CubeIDE performs:

- Compilation of `.cpp` files
- Linking of startup code and libraries
- Generation of output files

Output Files Generated

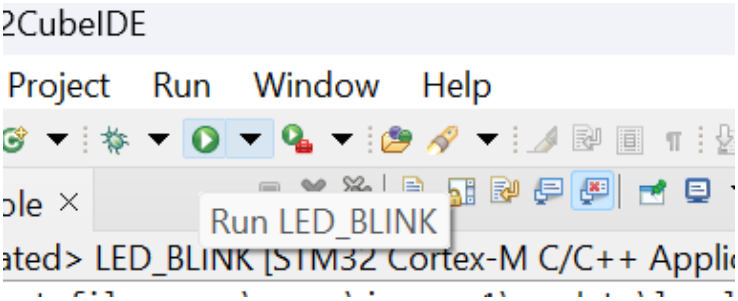
After successful build, you will see:

- `.elf` → Executable file (used for debugging)
- `.bin` / `.hex` → Binary file (used for flashing)



Step 9: Connect STM32F407 Discovery Board and Run

Flash the compiled program to the board and verify LED blinking.



1. Connect the **STM32F407 Discovery Board** to your PC using a **USB cable (ST-LINK port)**

2. Ensure:
   - Power LED is ON
   - Board is detected by the system

Flash & Run

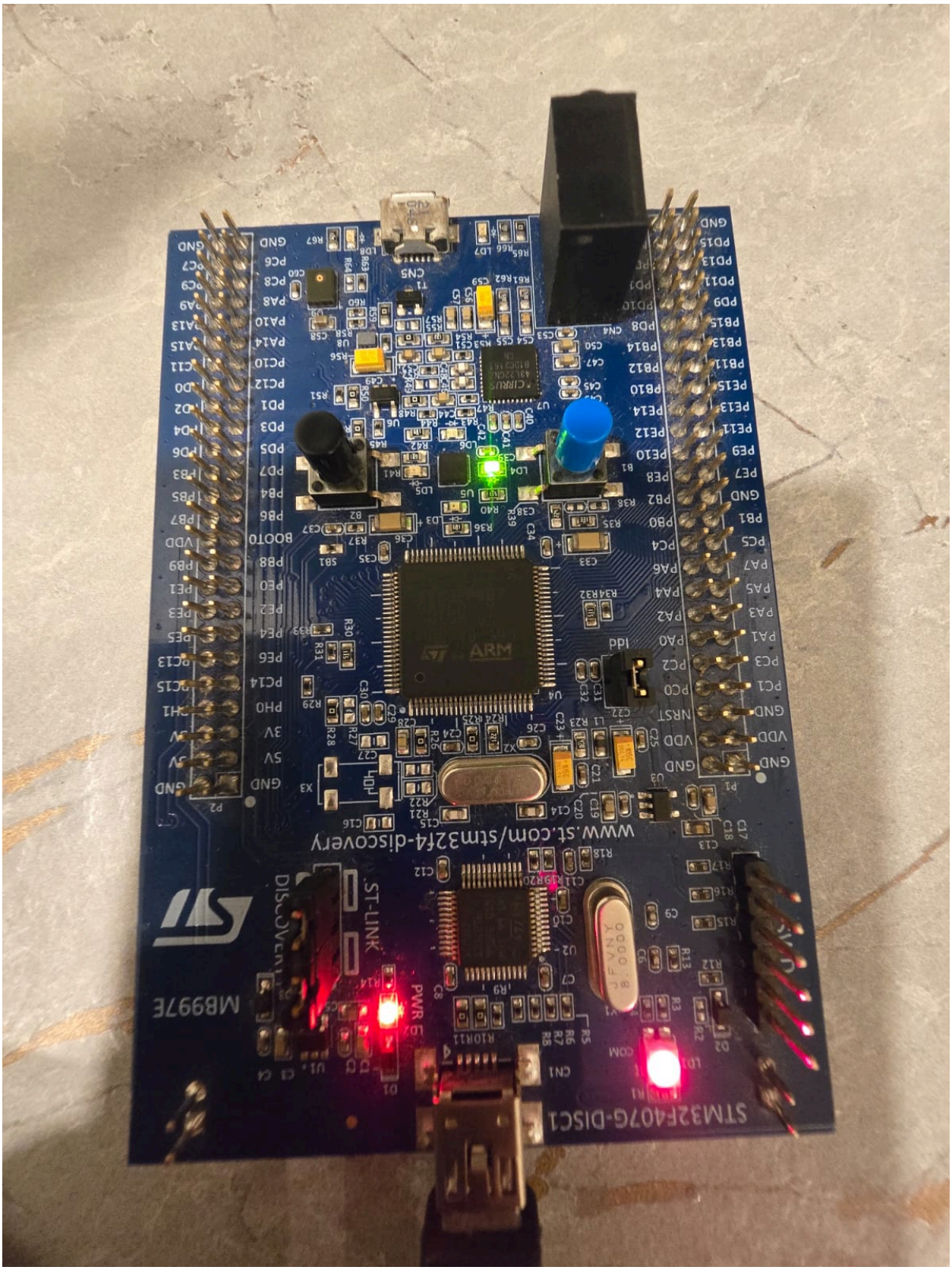
1. In IDE, click:
   👉 **Run (▶️ button)**
   OR
   👉 Right-click project → **Run As** → **STM32 Cortex-M C/C++ Application**
2. STM32CubeIDE will:
   - Compile (if needed)
   - Flash the `.elf` file to the board
   - Start execution

Expected Output

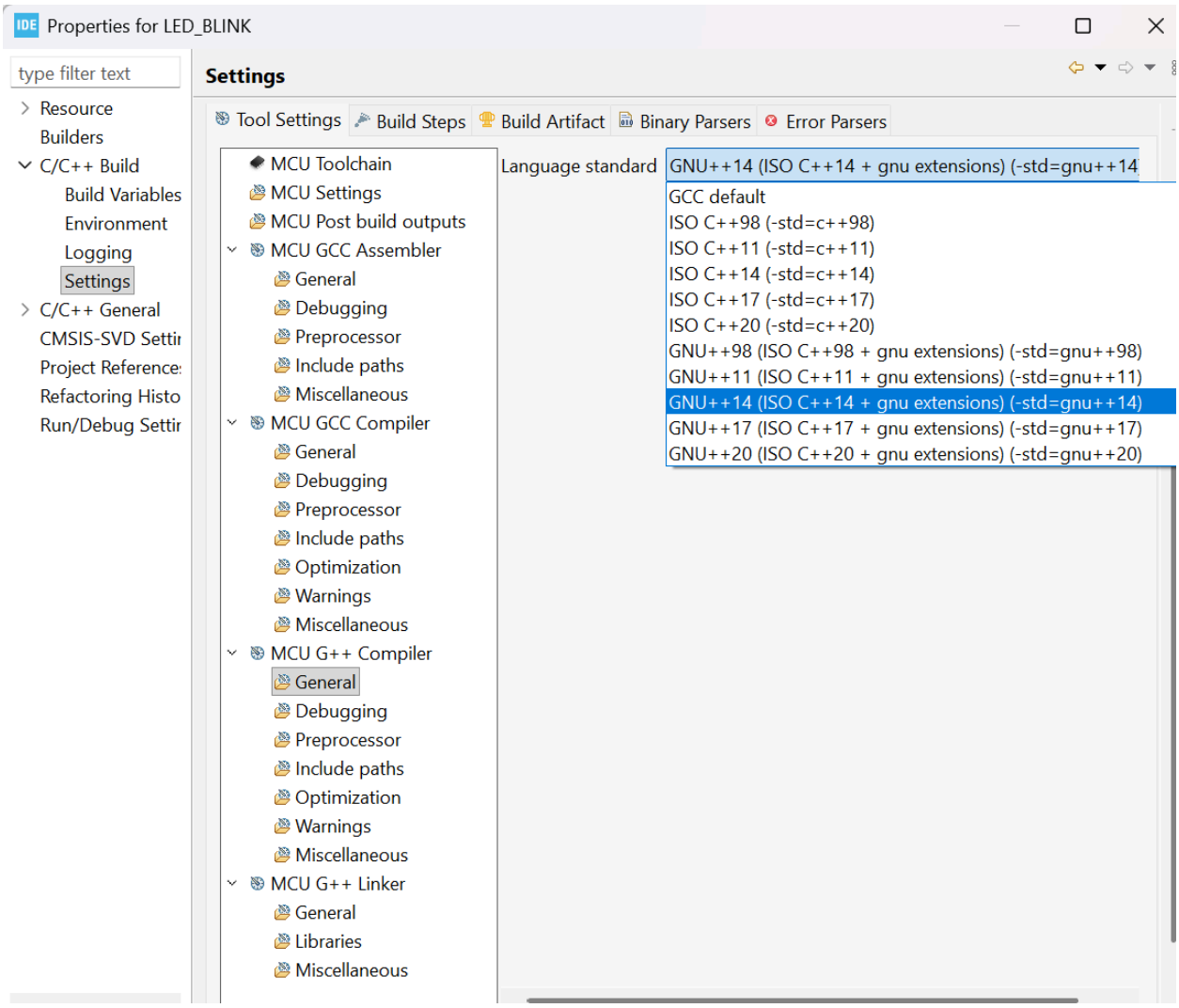
👉 The **Green LED (PD12)** on the board should:

- Turn ON and OFF continuously
- Blink at a visible rate (based on delay)

Output


STM32F407G-DISC1
www.st.com/stm32f4-discovery
ST-LINK
PWR
ARM

10. Configuring C++ Standard in STM32CubeIDE



Steps

1. **Right-click on Project**
2. Click → **Properties**
3. Navigate to:

```
C/C++ Build → Settings → MCU G++ Compiler → General
```

4. Locate:
   Language Standard

Default Setting

By default:

```
GNU++14 (C++14 with GNU extensions)
```

✔️ Stable

✔️ Supported across embedded toolchains

✔️ Good balance for embedded systems

Available Options

You can change to:

- **C++11** → Very stable, minimal features
- **C++14** → Default (recommended baseline)
- **C++17** → More features, still safe
- **C++20** → Advanced (use cautiously in embedded)

Important Notes for Embedded C++

1. Embedded Constraints

Newer C++ ≠ Always better in embedded systems

Limited Resources

- Microcontrollers have **limited Flash and RAM**
- Advanced C++ features may:
  - Increase code size

- Use more memory

Always write **lightweight and efficient code**

Compiler Support Varies

- Embedded toolchains may not fully support:
  - Latest C++ standards (C++20, C++23)
- Behavior may differ across compilers

Stick to **well-supported standards (C++14 / C++17)**

Avoid Heavy STL Usage

- Libraries like:
  - `vector`, `map`, `string`, `iostream`
- Can:
  - Increase binary size
  - Use dynamic memory (heap)
  - Reduce predictability

Prefer:

- Static memory
- Simple data structures
- Bare-metal control

In embedded systems, simplicity, predictability, and efficiency are more important than using the latest language features.

Complete Embedded C++ Code (Class-Based GPIO – PD12 Blink)

https://onlinegdb.com/CPvV9NUDZ

```
#include <cstdint>

//  Register addresses
constexpr std::uint32_t RCC_AHB1ENR_ADDR = 0x40023830;
constexpr std::uint32_t GPIOD_MODER_ADDR = 0x40020C00;
constexpr std::uint32_t GPIOD_ODR_ADDR   = 0x40020C14;

//  Register access (references)
inline volatile std::uint32_t& RCC_AHB1ENR =
    *reinterpret_cast<volatile std::uint32_t*>(RCC_AHB1ENR_AD
DR);

inline volatile std::uint32_t& GPIOD_MODER =
    *reinterpret_cast<volatile std::uint32_t*>(GPIOD_MODER_AD
DR);

inline volatile std::uint32_t& GPIOD_ODR =
    *reinterpret_cast<volatile std::uint32_t*>(GPIOD_ODR_ADD
R);

//  GPIO Class
class GPIO
{
public:
static void enable_clock()
    {
RCC_AHB1ENR |= (1U<<3);// Enable GPIOD clock
    }

static void set_output(std::uint32_t pin)
    {
GPIOD_MODER &=~ (3U<< (pin*2));// Clear mode bits
GPIOD_MODER |=  (1U<< (pin*2));// Set output mode
    }
```

```
static void toggle(std::uint32_t pin)
    {
GPIOD_ODR ^= (1U<<pin);// Toggle pin
    }
};

// Delay function
void delay()
{
for(volatile int i =0;i<100000;++i);
}

//  Entry point (important for C++)
int main(void)
{
    GPIO::enable_clock();// Step 1: Enable clock
    GPIO::set_output(12);// Step 2: Configure PD12

while (true)
    {
        GPIO::toggle(12);// Step 3: Toggle LED
delay();
    }
}
```

This project uses C++17 to enable inline variables, allowing clean and safe register mapping without multiple definition issues.

If inline variables are not used, C++14 is sufficient for most embedded applications.

Clean abstraction

- Hardware still accessed directly

- But wrapped in meaningful functions

Reusable

```
C++
// Example usage
GPIO::set_output(13);
GPIO::toggle(14);
```

Works for any GPIO pin without rewriting code

Type-safe + readable

- No macros
- No magic numbers scattered

This design is an example of zero-cost abstraction, where C++ improves code structure without increasing runtime overhead.

Modern Embedded C++ enables clean, maintainable, and type-safe designs while preserving full control over hardware and ensuring zero runtime overhead.

Note:

Prefer compile-time constants and minimal runtime overhead when designing embedded C++ drivers.

11. Final Conclusion

In this guide, we moved beyond simply blinking an LED and explored what actually happens inside the microcontroller.

We understood:

- How STM32 architecture connects CPU, buses, and peripherals
- How clocks flow from HSI/HSE through RCC to GPIO

- Why enabling RCC is mandatory before using any peripheral
- How to configure GPIO registers at the bare-metal level
- How to control hardware directly using Embedded C++